DA-2 Assessment Question

Subject: Application Development and Deployment Architecture

ANISHA SINGH

22MIC0020

1. A company is developing an **Online Food Ordering Application** using Java.
The development team is using **Maven** for project build and dependency management and **JUnit 5** for unit testing.

Task

1. Create a **Maven Project** for the application.
2. Add **JUnit 5 dependency** in pom.xml.
3. Create a Java class OrderService with the method:

   calculateTotal(double price, int quantity)

4. Write **JUnit 5 test cases** to test the above method.
5. Run the test cases using **Maven command**.
6. Show the **Maven project folder structure**.

Show the **console output after running test cases**

OrderService.java

pom.xml | OrderService.java | OrderServiceTest.java | Untitled-1

src > main > java > com > foodapp > OrderService.java > OrderService > calculateTotal(double, int)

```java
package com.foodapp;

/**
 * Service class for managing food orders
 */
public class OrderService {

    /**
     * Calculates the total cost of an order
     *
     * @param price Price per item
     * @param quantity Number of items ordered
     * @return Total cost (price * quantity)
     * @throws IllegalArgumentException if price or quantity is negative
     */
    public double calculateTotal(double price, int quantity) {
        if (price < 0) {
            throw new IllegalArgumentException(s: "Price cannot be negative");
        }
        if (quantity < 0) {
            throw new IllegalArgumentException(s: "Quantity cannot be negative");
        }
        return price * quantity;
    }
}
```

OrderServiceTest.java

```
OPEN EDITORS 1 unsaved
● pom.xml
  OrderService.java s...
× OrderServiceTest.ja...
  Untitled-1
FOOD-...
src
  main\java\com\foo...
    OrderService.java
  test\java\com\food...
    OrderServiceTest.java
target
pom.xml
OUTLINE
TIMELINE
SERVERS
LOGICAL STRUCTURE
JAVA PROJECTS
MAVEN
```

src > test > java > com > foodapp > OrderServiceTest.java > {} com.foodapp

```java
package com.foodapp;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import static org.junit.jupiter.api.Assertions.*;

/**
 * Test class for OrderService
 */
@DisplayName("OrderService Test Cases")
class OrderServiceTest {

    private OrderService orderService;

    @BeforeEach
    void setUp() {
        orderService = new OrderService();
    }

    @Test
    @DisplayName("Test calculateTotal with positive values")
    void testCalculateTotalWithPositiveValues() {
        double price = 150.0;
        int quantity = 3;
        double expected = 450.0;

        double actual = orderService.calculateTotal(price, quantity);

        assertEquals(expected, actual, 0.001,
            "Total should be price multiplied by quantity");
    }

    @Test
```

Pom.xml with JUnit5 dependencies

```
EXPLORER ...
OPEN EDITORS 1 unsaved
  pom.xml
  OrderService.java s...
  OrderServiceTest.ja...
  Untitled-1
FOOD-ORDERING-APP
  src
    main\java\com\foo...
      OrderService.java
    test\java\com\food...
      OrderServiceTest.java
  target
  pom.xml
```

```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-api</artifactId>
        <version>${junit.version}</version>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-engine</artifactId>
        <version>${junit.version}</version>
        <scope>test</scope>
    </dependency>


    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-params</artifactId>
        <version>${junit.version}</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

mvn compile

```
PS C:\Users\justx\Downloads\food-ordering-app> mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< com.foodapp:food-ordering-app >--------------------
[INFO] Building Food Ordering Application 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ food-ordering-app ---
[INFO] skip non existing resourceDirectory C:\Users\justx\Downloads\food-ordering-app\src\main\resources
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ food-ordering-app ---
[INFO] Changes detected - recompiling the module! :input tree
[INFO] Compiling 1 source file with javac [debug target 11] to target\classes
[WARNING] system modules path not set in conjunction with -source 11
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.418 s
[INFO] Finished at: 2026-02-14T16:59:05+05:30
[INFO] ------------------------------------------------------------------------
PS C:\Users\justx\Downloads\food-ordering-app>
```

mvn test

```
PS C:\Users\justx\Downloads\food-ordering-app> mvn test
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-java5/3.1.2/common-java5-3.1.2.jar (1
8 kB at 74 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.9.2/junit-platform-launch
er-1.9.2.jar (169 kB at 464 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.10.0/junit-platform-laun
cher-1.10.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.10.0/junit-platform-launc
her-1.10.0.pom (3.0 kB at 33 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.10.0/junit-platform-laun
cher-1.10.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.10.0/junit-platform-launc
her-1.10.0.jar (184 kB at 519 kB/s)
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.foodapp.OrderServiceTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.260 s -- in com.foodapp.OrderServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  9.809 s
[INFO] Finished at: 2026-02-14T17:00:35+05:30
[INFO] ------------------------------------------------------------------------
PS C:\Users\justx\Downloads\food-ordering-app>
```

mvn clean test

```
PS C:\Users\justx\Downloads\food-ordering-app> mvn clean test
[INFO] Compiling 1 source file with javac [debug target 11] to target\classes
[WARNING] system modules path not set in conjunction with -source 11
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ food-ordering-app ---
[INFO] skip non existing resourceDirectory C:\Users\justx\Downloads\food-ordering-app\src\test\resources
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ food-ordering-app ---
[INFO] Changes detected - recompiling the module! :dependency
[INFO] Compiling 1 source file with javac [debug target 11] to target\test-classes
[WARNING] system modules path not set in conjunction with -source 11
[INFO]
[INFO] --- surefire:3.1.2:test (default-test) @ food-ordering-app ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.foodapp.OrderServiceTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.248 s -- in com.foodapp.OrderServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.941 s
[INFO] Finished at: 2026-02-14T17:01:50+05:30
[INFO] ------------------------------------------------------------------------
```

mvn test -X

```
[DEBUG] Fork Channel [1] connected to the client.
[INFO] Running com.foodapp.OrderServiceTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.226 s -- in com.foodapp.OrderServiceTest
[DEBUG] Closing the fork 1 after saying GoodBye.
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.769 s
[INFO] Finished at: 2026-02-14T17:02:31+05:30
[INFO] ------------------------------------------------------------------------
[DEBUG] Shutting down adapter factory; available factories [file-lock, rwlock-local, semaphore-local, noop]; available name mappers
[discriminating, file-gaecv, file-gav, file-hgaecv, file-hgav, file-static, gaecv, gav, static]
[DEBUG] Shutting down 'file-lock' factory
[DEBUG] Shutting down 'rwlock-local' factory
[DEBUG] Shutting down 'semaphore-local' factory
[DEBUG] Shutting down 'noop' factory
```

mvn test -Dtest=OrderServiceTest

```
PS C:\Users\justx\Downloads\food-ordering-app> mvn test -Dtest=OrderServiceTest
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ food-ordering-app ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ food-ordering-app ---
[INFO] skip non existing resourceDirectory C:\Users\justx\Downloads\food-ordering-app\src\test\resources
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ food-ordering-app ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- surefire:3.1.2:test (default-test) @ food-ordering-app ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.foodapp.OrderServiceTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.233 s -- in com.foodapp.OrderServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.892 s
[INFO] Finished at: 2026-02-14T17:03:12+05:30
[INFO] ------------------------------------------------------------------------
PS C:\Users\justx\Downloads\food-ordering-app>
```

mvn install -DskipTests

```
PS C:\Users\justx\Downloads\food-ordering-app> mvn install -DskipTests
[INFO] --- resources:3.3.1:resources (default-resources) @ food-ordering-app ---
[INFO] skip non existing resourceDirectory C:\Users\justx\Downloads\food-ordering-app\src\main\resources
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ food-ordering-app ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ food-ordering-app ---
[INFO] skip non existing resourceDirectory C:\Users\justx\Downloads\food-ordering-app\src\test\resources
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ food-ordering-app ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- surefire:3.1.2:test (default-test) @ food-ordering-app ---
[INFO] Tests are skipped.
[INFO]
[INFO] --- jar:3.4.1:jar (default-jar) @ food-ordering-app ---
[INFO] Building jar: C:\Users\justx\Downloads\food-ordering-app\target\food-ordering-app-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- install:3.1.2:install (default-install) @ food-ordering-app ---
[INFO] Installing C:\Users\justx\Downloads\food-ordering-app\pom.xml to C:\Users\justx\.m2\repository\com\foodapp\food-ordering-app\
1.0-SNAPSHOT\food-ordering-app-1.0-SNAPSHOT.pom
[INFO] Installing C:\Users\justx\Downloads\food-ordering-app\target\food-ordering-app-1.0-SNAPSHOT.jar to C:\Users\justx\.m2\reposi
ory\com\foodapp\food-ordering-app\1.0-SNAPSHOT\food-ordering-app-1.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.840 s
[INFO] Finished at: 2026-02-14T17:03:51+05:30
```

2. Technical Stack / Framework: SonarQube

A software company is developing a **Java-based application**.
To ensure **code quality and security**, the development team decides to use **SonarQube** for code analysis.

Task to be Performed:

1. Create a simple **Java program** with:
   - Include one intentional coding mistake (example: unused variable, duplicate code, or bad naming)
2. Install and configure **SonarQube Server**.
3. Run **SonarQube code analysis** on the Java program.
4. View the **SonarQube Dashboard Report**.
5. Identify the following from SonarQube report:
   - Code Smells
   - Bugs
   - Vulnerabilities
6. Fix at least one issue reported by SonarQube.
7. Re-run analysis and compare results before and after fixing.

Java source code

```
public class BankingApplication {

    // Unused variable - Code Smell
    private static int unusedCounter = 0;

    // Bad naming convention - Code Smell
    private String n;

    public BankingApplication(String n) {
        this.n = n;
    }

    // Duplicate code - Code Smell
    public double calculateInterest(double principal, double rate, int
years) {
        double interest = principal * rate * years / 100;
        System.out.println("Interest calculated: " + interest);
        return interest;
    }

    // Duplicate code - Code Smell
    public double calculateCompoundInterest(double principal, double
rate, int years) {
        double interest = principal * rate * years / 100;  // Same
calculation as above
        System.out.println("Compound Interest calculated: " +
interest);
        return interest;
    }

    // Potential bug - Division by zero not handled
    public double divide(int a, int b) {
        return a / b;  // No check for b == 0
    }

    // Security vulnerability - Hardcoded password
    public boolean authenticate(String username, String password) {
        String hardcodedPassword = "admin123";  // Hardcoded credential
        return password.equals(hardcodedPassword);
```

```
    }

    // Code smell - Empty catch block
    public void processTransaction(String data) {
        try {
            Integer.parseInt(data);
        } catch (NumberFormatException e) {
            // Empty catch block - swallowing exception
        }
    }

    // Main method for demonstration
    public static void main(String[] args) {
        BankingApplication app = new BankingApplication("MyBank");

        // Test calculations
        double simpleInterest = app.calculateInterest(10000, 5, 3);
        double compoundInterest = app.calculateCompoundInterest(10000,
5, 3);

        // Test authentication
        boolean isAuthenticated = app.authenticate("admin",
"admin123");
        System.out.println("Authentication: " + isAuthenticated);

        // Test division
        double result = app.divide(100, 5);
        System.out.println("Division result: " + result);

        // Test exception handling
        app.processTransaction("invalid");

        System.out.println("Application executed successfully");
    }
}
```

SonarQube cloud

My Projects | My Issues | Explore

Analyze projects

Manual setup is not recommended, and leads to missing features like appropriate setup of your project or analysis feedback in the Pull Request. We recommend to import your projects

Organization

anisha

Create another organization

Display Name *

Banking Application

Up to 255 characters

Project Key *

banking-application

Up to 400 characters. All letters, digits, dash, underscore, period or colon.

Project visibility *

- Public
  Anyone will be able to browse your source code and see the result of your analysis.
- Private
  Only members of the organization will be able to browse your source code and see the result of your analysis.

Next

Project onboarding

Analyze from your local sources

1 Run analysis on your project

What option best describes your project?

JS/TS & Web | Maven | Gradle | .NET | Python | C, C++ or ObjC | Flutter or Dart | **Other (for Go, PHP, ...)**

Which OS do you run your build on?

Linux | **Windows** | macOS

Download and unzip the SonarScanner for Windows

And add the `bin` directory to the `%PATH%` environment variable

Download

Download and unzip the SonarScanner for Windows

And add the `bin` directory to the `%PATH%` environment variable

Download

Configure the `SONAR_TOKEN` environment variable

1. Name of the environment variable: `SONAR_TOKEN`
2. Value of the environment variable: `9d7644ed2c2a2684cec6c480dab53ac553bcfa22`

Execute the SonarScanner from your computer

Run the following command in your project's folder.

```
sonar-scanner.bat `
  -D"sonar.organization=uneazee" `
  -D"sonar.projectKey=banking-application"
```

Copy

Please visit the SonarScanner documentation for more details.

run sonar-scanner.bat

EXPLORER

OPEN EDITORS
- BankingApplic... 4
- BankingApplic... 1
- sonar-project.prop...

SONARQUBE-DEMO
- .scannerwork
- bin
- src
  - BankingApplicat... 4
  - BankingApplicati... 1
- sonar-project.properties

OUTLINE
TIMELINE
SERVERS
LOGICAL STRUCTURE

BankingApplication.java 4 | BankingApplicationFixed.java 1 | sonar-project.properties

sonar-project.properties

```
# Project identification
sonar.projectKey=banking-application
sonar.organization=uneazee

# SonarCloud URL
sonar.host.url=https://sonarcloud.io

# Authentication token
sonar.token=9d7644ed2c2a2684cec6c480dab53ac553bcfa22

# Path to source directories
sonar.sources=src

# Encoding of the source files
sonar.sourceEncoding=UTF-8

# Java version
sonar.java.source=11

# Tell SonarQube we don't have compiled binaries
```

PROBLEMS 5 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | QUERY RESULTS | GITLENS

```
PS C:\Users\justx\Downloads\sonarqube-demo> sonar-scanner.bat
INFO: Inferred api base url 'https://api.sonarcloud.io' from host url 'https://sonarcloud.io'.
INFO: Create analysis
INFO: Create analysis (done) | time=346ms
INFO: Inferred api base url 'https://api.sonarcloud.io' from host url 'https://sonarcloud.io'.
INFO: Inferred api base url 'https://api.sonarcloud.io' from host url 'https://sonarcloud.io'.
INFO: Load active rules
INFO: Load active rules (done) | time=1825ms
INFO: Organization key: uneazee
INFO: Preprocessing files...
INFO: 1 language detected in 2 preprocessed files (done) | time=225ms
INFO: 0 files ignored because of inclusion/exclusion patterns
INFO: 0 files ignored because of scm ignore settings
INFO: Loading plugins for detected languages
INFO: Load/download plugins
```

```
PS C:\Users\justx\Downloads\sonarqube-demo> sonar-scanner.bat
INFO: Analysis report generated in 479ms, dir size=343 KB
INFO: Analysis report compressed in 88ms, zip size=68 KB
INFO: Analysis report uploaded in 656ms
INFO: ANALYSIS SUCCESSFUL, you can find the results at: https://sonarcloud.io/dashboard?id=banking-application
INFO: Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
INFO: More about the report processing at https://sonarcloud.io/api/ce/task?id=AZxc3tRh9RL4PIzrvhUF
INFO: ------------- Upload SCA dependency files
INFO: Sensor cache published successfully
INFO: Analysis total time: 2:06.511 s
INFO: ------------------------------------------------------------------------
INFO: EXECUTION SUCCESS
INFO: ------------------------------------------------------------------------
INFO: Total time: 3:09.313s
INFO: Final Memory: 59M/214M
INFO: ------------------------------------------------------------------------
PS C:\Users\justx\Downloads\sonarqube-demo>
```

anisha / Banking Application / Summary

Summary

master

Private • 96 Lines of Code • Last analysis 5 minutes ago • 6429a53a

Bind project

Security	Reliability	Maintainability
0 Open issues A	1 Open issues B	24 Open issues B

Accepted Issues	Coverage	Duplications
0	0.0% No conditions set on 61 Lines to cover	0.0% No conditions set on 159 Lines

Security Hotspots

2

Measures

master

- Project Overview
- Security
- Reliability
  - Overview
  - Overall Code
    - Bugs 1
    - Rating B
    - Remediation Effort 5min

Banking Application — View as List — 2 files

Reliability Rating B — See history

src/BankingApplication.java	B
src/BankingApplicationFixed.java	A

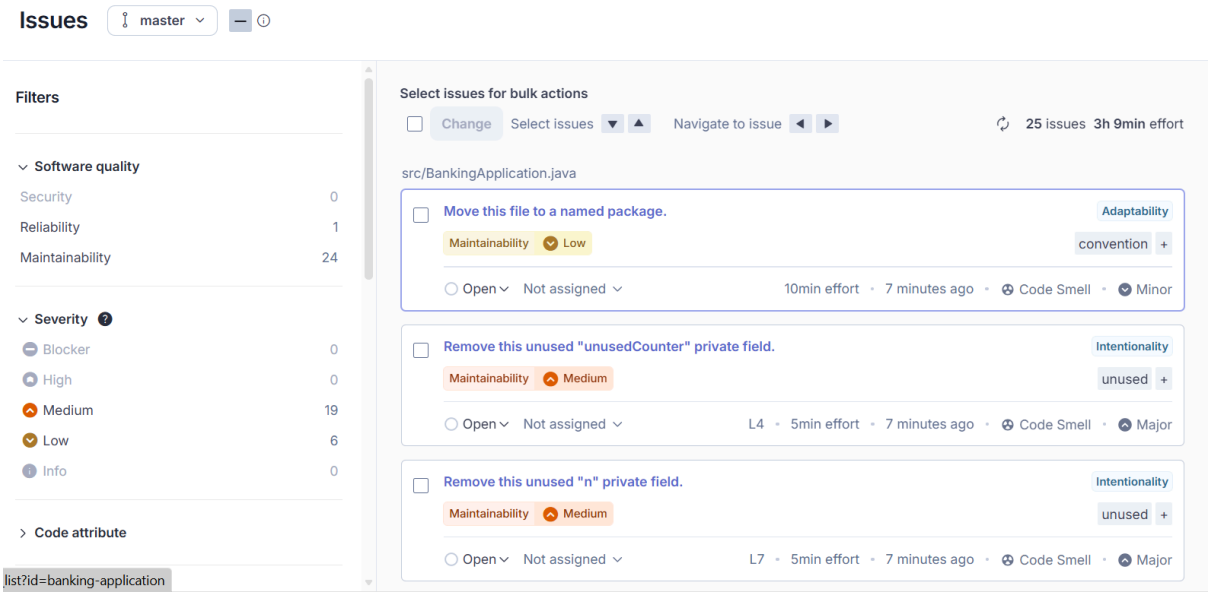
2 of 2 shown

© 2018-2026 SonarSource Sàrl. All rights reserved.

Terms Pricing Privacy Cookie Policy Security Community Documentation Contact us Status About

Maintainability
Overview
Overall Code
Code Smells 24
Debt 3h 4min
Debt Ratio 6.4%
Rating B
Effort to Reach A 40min

Issues


Issues
master
Filters
Software quality
Security 0
Reliability 1
Maintainability 24
Severity
Blocker 0
High 0
Medium 19
Low 6
Info 0
Code attribute
list?id=banking-application
Select issues for bulk actions
Change
Select issues
Navigate to issue
25 issues 3h 9min effort
src/BankingApplication.java
Move this file to a named package.
Adaptability
Maintainability Low
convention
Open
Not assigned
10min effort • 7 minutes ago • Code Smell • Minor
Remove this unused "unusedCounter" private field.
Intentionality
Maintainability Medium
unused
Open
Not assigned
L4 • 5min effort • 7 minutes ago • Code Smell • Major
Remove this unused "n" private field.
Intentionality
Maintainability Medium
unused
Open
Not assigned
L7 • 5min effort • 7 minutes ago • Code Smell • Major

Java source code fixing

```
public class BankingApplicationFixed {

    // Removed unused variable
    // Improved naming convention
    private String bankName;

    public BankingApplicationFixed(String bankName) {
        this.bankName = bankName;
    }

    // Refactored to remove duplicate code
    public double calculateSimpleInterest(double principal, double
rate, int years) {
        return calculateInterestBase(principal, rate, years);
    }

    // Uses compound interest formula
    public double calculateCompoundInterest(double principal, double
rate, int years) {
        double amount = principal * Math.pow(1 + (rate / 100), years);
        return amount - principal;
    }

    // Base calculation method to avoid duplication
    private double calculateInterestBase(double principal, double rate,
int years) {
        double interest = principal * rate * years / 100;
        System.out.println("Interest calculated: " + interest);
        return interest;
    }

    // Fixed - Added division by zero check
    public double divide(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException("Cannot divide by zero");
        }
        return (double) a / b;
    }

    // Fixed - Removed hardcoded password, should use secure
authentication
```

```java
    public boolean authenticate(String username, String password) {
        // In production, use proper authentication service
        // This is just for demonstration
        if (username == null || password == null) {
            return false;
        }
        // Call external authentication service here
        return true;  // Placeholder
    }

    // Fixed - Proper exception handling
    public void processTransaction(String data) {
        try {
            Integer.parseInt(data);
        } catch (NumberFormatException e) {
            System.err.println("Invalid transaction data: " +
e.getMessage());
            // Log the error or take appropriate action
        }
    }

    // Main method for demonstration
    public static void main(String[] args) {
        BankingApplicationFixed app = new
BankingApplicationFixed("MyBank");

        // Test calculations
        double simpleInterest = app.calculateSimpleInterest(10000, 5,
3);
        double compoundInterest = app.calculateCompoundInterest(10000,
5, 3);

        System.out.println("Simple Interest: " + simpleInterest);
        System.out.println("Compound Interest: " + compoundInterest);

        // Test authentication
        boolean isAuthenticated = app.authenticate("admin",
"secure_password");
        System.out.println("Authentication: " + isAuthenticated);

        // Test division
        try {
            double result = app.divide(100, 5);
```

```
            System.out.println("Division result: " + result);

            // This will throw exception
            app.divide(100, 0);
        } catch (ArithmeticException e) {
            System.err.println("Error: " + e.getMessage());
        }

        // Test exception handling
        app.processTransaction("invalid");

        System.out.println("Application executed successfully");
    }
}
```

Rerun after fixing

src/BankingApplication.java

See all issues in this file

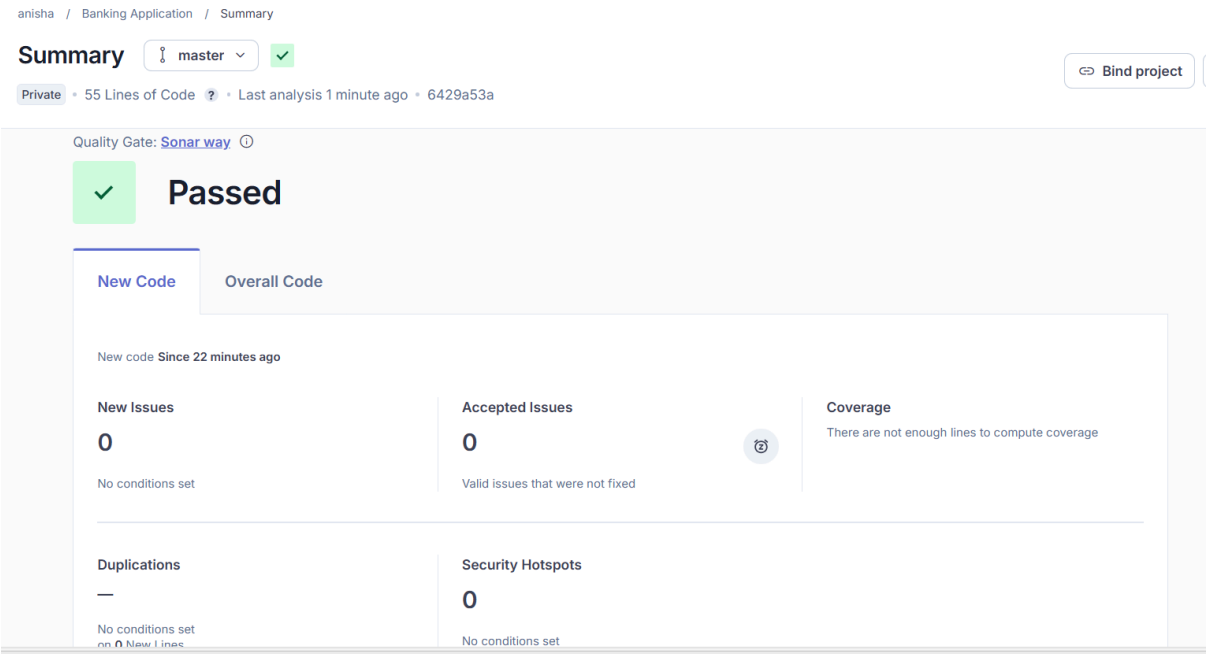
```
1  ...  public class BankingApplication {
2  ...
3  ...      // Unused variable - Code Smell
4  ...      private static int unusedCounter = 0;
```

Remove this unused "unusedCounter" private field.

```
PS C:\Users\justx\Downloads\sonarqube-demo> sonar-scanner.bat
INFO: * Protobuf reading complete | memory total=644 | free=308 | used=335 (MB)
INFO: * Build architecture.graph.java.namespace_graph.default_perspective hierarchy graph complete (filtered=false) | memory total=6
05 | used=338 (MB)
INFO: Sensor JavaArchitectureSensor [architecture] (done) | time=306ms
INFO: Sensor Zero Coverage Sensor
INFO: Sensor Zero Coverage Sensor (done) | time=10ms
INFO: Sensor Java CPD Block Indexer
INFO: Sensor Java CPD Block Indexer (done) | time=27ms
INFO: ------------- Gather SCA dependencies on project
INFO: Checking if SCA is enabled for organization uneazee
INFO: Dependency analysis skipped
INFO: CPD Executor Calculating CPD for 2 files
INFO: CPD Executor CPD calculation finished (done) | time=14ms
INFO: SCM Publisher SCM provider for this project is: git
INFO: SCM Publisher 2 source files to be analyzed
INFO: SCM Publisher 0/2 source files have been analyzed (done) | time=260ms
WARN: Missing blame information for the following files:
WARN:   * src/BankingApplication.java
WARN:   * src/BankingApplicationFixed.java
WARN: This may lead to missing/broken features in SonarCloud
INFO: Analysis report generated in 272ms, dir size=344 KB
INFO: Analysis report compressed in 49ms, zip size=69 KB
INFO: Analysis report uploaded in 864ms
INFO: ANALYSIS SUCCESSFUL, you can find the results at: https://sonarcloud.io/dashboard?id=banking-application
INFO: Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
INFO: More about the report processing at https://sonarcloud.io/api/ce/task?id=AZxc65iu9RL4PIzrvhZf
INFO: ------------- Upload SCA dependency files
INFO: Sensor cache published successfully
INFO: Analysis total time: 53.241 s
INFO: ------------------------------------------------------------------------
INFO: EXECUTION SUCCESS
INFO: ------------------------------------------------------------------------
INFO: Total time: 57.160s
INFO: Final Memory: 59M/210M
INFO: ------------------------------------------------------------------------
PS C:\Users\justx\Downloads\sonarqube-demo>
```

Do you want
extension fro

Fixed all issues !



Short Note on SonarQube usage in real-world software development

- **Catches Issues Automatically:** SonarQube runs in the background during your build process (through tools like Jenkins or GitHub Actions) and flags bugs, security problems, and messy code before it gets deployed.
- **Stops Problems Early:** It finds things like potential crashes, security holes (SQL injection, hardcoded passwords), and other issues that would be expensive and embarrassing to fix later in production.
- **Acts as a Gatekeeper:** Teams set up "quality gates" that basically say "if your code has critical bugs, you can't merge it." This keeps bad code from sneaking into the main codebase.
- **Makes Code Reviews Easier:** Instead of reviewers pointing out formatting issues and obvious bugs, SonarQube handles that automatically. Reviewers can focus on whether the code actually solves the problem well.
- **Helps with Security and Compliance:** Companies need to meet certain standards (like GDPR or HIPAA), and SonarQube helps by detecting common security vulnerabilities and keeping records of code quality.
- **Manages Technical Debt:** It tracks how messy your codebase is getting over time and points out areas that need cleanup, which helps teams plan refactoring work.
- **Gives Instant Feedback:** Plugins like SonarLint work right in your code editor, so you see problems as you type rather than finding out later.
- **Has Limitations:** It won't catch everything - it can't test your logic or run your code. Sometimes it flags things that aren't really problems, so you need to configure it properly.

Most development teams find SonarQube really valuable once they get it set up. It's basically like having someone double-check your work automatically, which means fewer bugs and better code overall.